

22. A Lagrange-interpoláció hibája, interpoláció a MATLAB-ban

A Lagrange-interpoláció hibája

Az interpoláló polinom az alappontokban pontosan illeszkedik, de közöttük eltérhet az eredeti függvénytől — ezt nevezzük **interpolációs hibának**. A hibavizsgálathoz feltesszük, hogy f az alappontokat és az x pontot tartalmazó legszűkebb $[a, b]$ intervallumon $(n+1)$ -szer differenciálható. Ekkor az $\omega_n(x) = (x-x_0)(x-x_1)\dots(x-x_n)$ jelöléssel a maradéktag:

$$R(x) = f(x) - P(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

A Lagrange-interpoláció maradéktagja

$$f(\hat{x}) - p_n(\hat{x}) = \frac{\omega_n(\hat{x})}{(n+1)!} f^{(n+1)}(\xi), \quad \xi \in [a, b].$$

ahol $f^{(n+1)}(\xi)$ az eredeti függvény $(n+1)$ -edik deriváltja egy $\xi \in [x_0, x_n]$ közbülső pontban. A képlet a Rolle-tétel ismételt alkalmazásával vezethető le (egy ügyesen választott segédfüggvénynek legalább $n+2$ zérushelye van). Ha $f^{(n+1)}$ korlátos, felső korlátja μ_{n+1} , akkor:

$$|f(\hat{x}) - p_n(\hat{x})| \leq \frac{|\omega_n(\hat{x})|}{(n+1)!} \mu_{n+1} \leq \frac{\max_{x \in [a, b]} |\omega_n(x)|}{(n+1)!} \mu_{n+1}.$$

A hiba tehát három dologtól függ: az $(n+1)$ -edik derivált nagyságától, az x alappontoktól mért távolságtól, valamint az alappontok számától és elhelyezkedésétől.

Runge-jelenség: egyenletes (ekvidisztáns) alappontoknál magas fokszámon a polinom a tartomány **szélein** erősen oszcillálhat. A hiba csökkenthető **Csebisev-pontok** használatával, amelyek a $\max |\omega_n(x)|$ értéket minimalizálják, és így letompítják a szélső oszcillációkat.

Interpoláció a MATLAB-ban

Egy dimenzióban a fő eszköz az **interp1**:

- $y0 = \text{interp1}(x, y, x0, 'típus')$ — az (x, y) pontokra illeszt és kiértékel $x0$ -ban (az x -nek szigorúan növekvőnek kell lennie).
- A típus lehet 'linear' (alapértelmezett), 'nearest', 'cubic' vagy 'spline'.

A 'nearest' durva, lépcsős közelítést ad, a 'spline' viszont sima, kiváló leírást — a lineáris is csak az erősen görbült részekben tér el. (Magasabb dimenzióban: **interp2**, illetve szórt adatra **griddata**, gyakran **meshgrid-del**.) Az interpoláló polinom közvetlenül is előállítható a **polyfit** / **polyval** párossal.

```
matlab Kód másolása

% Adatpontok
x = [0 1 2];
y = exp(-x);

% Polinom illesztése
P = polyfit(x, y, length(x)-1);

% Polinom kiértékelése és hiba számítása
xx = linspace(0, 2, 100); % Sűrűbb pontok az ábrázoláshoz
yy = polyval(P, xx); % Interpolációs polinom kiértékelése
f_true = exp(-xx); % Eredeti függvény

% Hiba kiszámítása
error = abs(f_true - yy);

% Ábrázolás
figure;
subplot(2, 1, 1);
plot(xx, f_true, 'b', xx, yy, 'r--', x, y, 'ko');
legend('Eredeti függvény', 'Interpoláció', 'Adatpontok');
title('Interpoláció és eredeti függvény');
xlabel('x');
ylabel('y');
grid on;

subplot(2, 1, 2);
plot(xx, error, 'm');
title('Interpolációs hiba');
xlabel('x');
ylabel('Hiba');
grid on;
```

MATLAB-példa: interpoláció és hibavizsgálat

A polyfit egyszerű, de magas fokszámon (a rosszul kondicionált Vandermonde-mátrix miatt) instabil lehet — ilyenkor érdemes spline-t vagy Csebisev-alappontokat használni.